KOI CONTENT PACK · ACCEPTANCE TESTING

Test Guide

Nine tests that prove every part of the pack works on your tenant — and what a pass looks like for each.

9	~45	1	0
tests	minutes	tenant	risk to prod

REQUIREMENTS

What you need, by what you are deploying

The two deployments have very different prerequisites. Pick your column before you start.

Full pack

Collection, triage, investigation and response

- ✓ Cortex XSIAM tenant — the parsing and modeling rules are fromversion 8.4.0
- ✓ A KOI API key with the xt-Administrator role
- ✓ An egress IP KOI accepts, or a Cortex engine that has one
- ✓ demisto-sdk 1.38+ and a Standard XSIAM API key
- ✓ At least one ingested KOI alert, for tests 5 to 7

Script Runner only

Run KOI scripts on endpoints from a Job

- ✓ A Cortex XSIAM tenant with Cortex agents installed
- ✓ The KOI script package uploaded to Action Center → Scripts Library
- ✓ An endpoint group containing connected, unisolated agents
- ✓ Optionally a mail-sender instance for notifications
- ✓ No KOI API key and no KOI integration instance

The Script Runner playbooks call no koi- command at all — only core-get-scripts, core-get-endpoints and core-script-run.*

INSTALLATION

Choose your path — it decides whether events flow

Path	What lands	Events?	Use when
demisto-sdk --xsiam	Everything, at the version you were sent	✓ yes	The normal path
Pre-built dist/Koi.zip	Integration + 3 playbooks only. No rules, no dashboard	✗ no	Not for XSIAM
Manual per-item	Only the items you import yourself	✗ no	Partial adoption

! The pre-built zip was built for a different marketplace target: inside it the collector flag is isfetchevents: false, and it ships no parsing or modeling rules. Upload it to XSIAM and the commands work, but koi_koi_raw stays empty forever. Event collection needs a demisto-sdk --xsiam upload.

INSTALLATION

Full pack with demisto-sdk

Steps

1. Install the SDK — 1.38+, older versions reject the platform marketplace.
2. `pip3 install demisto-sdk`
3. Place the pack at Packs/Koi inside a content-repo scaffold.
4. Copy Packs/ApiModules/Scripts/ContentClientApiModule/ from demisto/content.
5. Point the SDK at the tenant:
6. `export DEMISTO_BASE_URL=https://api-<tenant>...`
7. `export DEMISTO_API_KEY=<standard-key>`
8. `export XSIAM_AUTH_ID=<key-id>`
9. `demisto-sdk upload -i Packs/Koi -z --xsiam`

What to expect

- ✓ The pack appears on the tenant at its version.
- ✓ Integration, playbooks, rules and dashboard all land together.
- ✓ Fetch events is available on the instance.

! The SDK sends the API key as-is, so it works only with a Standard XSIAM key. With an Advanced key the SDK cannot connect — build with zip-packs and POST the artifact to /xsoar/contentpacks/installed/upload with signed headers instead.

INSTALLATION

Script Runner only — the minimal import

To run KOI scripts from an XSIAM Job and nothing else, import these four items by hand. Nothing else in the pack is required.

1	**Koi Unified - Execute Endpoint Script** Playbooks → Import	Import first — the others reference it by name.
2	**Koi Unified - Process Config Entry** Playbooks → Import	Import second — calls the executor above.
3	**Koi Unified - Script Runner** Playbooks → Import	Import third — the Job entry point.
4	**Koi Script Runner** Object Setup → Lists, type JSON	The configuration array. Name must be exact.

Then create the Job

Automation → Jobs → New Job, Time-triggered, playbook Koi Unified - Script Runner. You do NOT need the KOI integration, an API key, the parsing or modeling rules, the dashboard, or any of the seven KOI triage playbooks — the Script Runner set calls no KOI API command.

REQUIREMENTS

Names that must match exactly

Everything below binds by name at run time. A mismatch does not raise a config error — it fails or silently skips at the next run.

Name or key	Must match	If it does not
`Koi Script Runner`	The JSON List name, character for character	Job runs, does nothing, prints "no valid configuration found"
`script.name`	A script in Action Center → Scripts Library	Fails: "script name was not found in the library"
`script.uuid`	Optional. Wins over script.name when set	Two library entries with one name fail as "multiple scripts matched"
`target.endpoint_groups`	An existing endpoint group with agents in it	Entry is SKIPPED — an info entry, no failure email
`target.endpoint_os`	Windows, macOS or Linux	Entry is reported invalid and skipped
`sendmail_instance.name`	An enabled mail instance, if notifications are on	Notification step fails
`Sub-playbook names`	KOI - / Koi Unified - names, unrenamed	Parent cannot resolve the sub-playbook

The List name is fixed in the playbook task — the platform cannot parameterise ${lists.<name>}. To use another name you must edit the task.

REQUIREMENTS

The Koi Script Runner List

The List is the entire configuration surface for the Job. It says which script to run, on which endpoints, and who to notify — so retargeting never means editing a playbook.

One entry per script + OS scope

```
[
  {
    "script": { "name": "KOI Deployment Script - Windows" },
    "target": {
      "endpoint_groups": ["KOI Endpoints"],
      "endpoint_os": "Windows"
    }
  },
  { ...one more entry per OS you deploy to... }
]
```

The three pointers you set

`script.name`

A script package in Action Center → Scripts Library

Sample: KOI Deployment Script - Windows

`target.endpoint_groups`

An endpoint group holding connected, unisolated agents

Sample: KOI Endpoints

`target.endpoint_os`

The OS scope for that entry

Windows, macOS or Linux

! There is no required script name. The pack ships neither the List nor any script package — you create both, and the names above are samples from the customer guide, not fixed values. What matters is that every name in the List matches something that exists on your tenant. To retarget later — a different script, another endpoint group, a new OS — edit the List; the playbooks never change.

REQUIREMENTS

Every List field, and what it defaults to

Field	Required	Default and notes
`script.name`	Yes *	No default. Must equal the Scripts Library name exactly.
`script.uuid`	No	Wins over script.name. Use it to survive renames or duplicates.
`script.polling_interval_in_seconds`	No	60 seconds.
`script.timeout_in_seconds`	No	1800 seconds.
`target.endpoint_os`	Yes	No default. Windows, macOS or Linux — case-insensitive.
`target.endpoint_groups`	One of these	No default. Group names holding the target agents.
`target.endpoint_hostnames`	One of these	No default. Specific hostnames instead of a group.
`notification.recipients`	No	Omit for no email at all.
`notification.sendmail_instance.name`	No	Omit to use any enabled mail instance.
`disabled`	No	false. Set true to skip the entry without deleting it.

* *script.name or script.uuid — one of the two is required. Every entry also needs endpoint_os, plus endpoint_groups or endpoint_hostnames.*

TEST PLAN

What you are going to prove

1

Connectivity & authorisation

The key works and your egress is accepted

2

Event collection

Alerts and audit logs land in koi_koi_raw

3

Command surface

The read commands return live tenant data

4

Dashboard & data model

Events are normalised and visualised

5

Alert triage end to end

Every alert is investigated and classified

6

The two investigations

Item and device context is gathered

7

Gated response

Nothing is blocked without a human

8

Scheduled hunt

Risky MCP servers surface without an alert

9

Script Runner job

KOI scripts run on Cortex-Agent endpoints

Run them in order — each test assumes the previous one passed.

TEST 1

Connectivity & authorisation

Steps

1. Settings → Integrations → add a KOI instance.
2. `Server URL:  https://api.prod.koi.security/`
3. Paste the API key, then click Test.
4. `!koi-devices-list limit=5`
5. `!koi-users-list limit=1`

What to expect

- ✓ Test returns Success.
- ✓ A device table with up to five rows.
- ✓ koi-users-list returns a user record.
- ✓ No 403 Forbidden on either command.

! koi-users-list is the egress probe. A valid key can still return 403 from a shared cloud egress — if it does, run the integration on a Cortex engine whose egress IP KOI accepts. Event collection is unaffected either way.

TEST 2

Event collection

Steps

1. On the instance, enable Fetch events and save.
2. Wait for one or two fetch cycles.
3. `dataset = koi_koi_raw`
4. `| comp count() as events by source_log_type`
5. Re-run the query after the next cycle.
6. `!koi-fetch-context-get`

What to expect

- ✓ Rows for the Alerts and Audit log types.
- ✓ Counts increase between the two runs.
- ✓ koi-fetch-context-get shows the cursor.

! Event collection uses a different KOI endpoint from the command surface, so it keeps working even when the sensitive endpoints return 403. A passing test 2 alongside a failing test 1 points squarely at egress.

TEST 3

Command surface

Steps

1. Sweep the read commands in the playground:
2. `!koi-inventory-list limit=5`
3. `!koi-koidex-search query=<package>`
4. `!koi-blocklist-get`
5. `!koi-policy-list`
6. `!koi-findings-list`
7. Take a device id from test 1 and run:
8. `!koi-device-inventory-get device_id=<id>`

What to expect

- ✓ Each returns a table plus context under Koi.*
- ✓ No 403 on any of them.
- ✓ Lists everything installed on that host.
- ✓ 500 items on one host, 35 flagged high risk.

! Twenty of the twenty-six commands are read-only. Only six change tenant state: the allowlist and blocklist add/remove pairs, koi-policy-status-update and koi-fetch-context-set. None of them run in this test.

TEST 4

Dashboard & data model

Steps

1. Dashboards & Reports → open Koi Alerts Dashboard.
2. Set the time range to cover your ingested data.
3. Confirm each widget renders.
4. Then check the modelling with XQL:
5. `dataset = koi_koi_raw | fields xdm.*`

What to expect

- ✓ Widgets render with data, not empty states.
- ✓ XDM fields are populated, not null.
- ✓ Alert activity matches what KOI shows.

! An empty dashboard with a passing test 2 usually means the time range, not the data. Widen it before investigating the rules.

TEST 5

Alert triage, end to end

Steps

1. Open an ingested KOI alert (source koi).
2. Attach and run KOI - Alert Triage.
3. Watch the Work Plan to completion.
4. Read the war room from top to bottom.

What to expect

- ✓ Work Plan completes — runStatus: completed.
- ✓ An item investigation summary is posted.
- ✓ A device investigation summary is posted.
- ✓ A triage summary carries a verdict.
- ✓ Benign auto-closes, Suspicious stays open, Malicious raises severity.

! Reference run: a vulnerable-dependency alert on a package whose KOI catalog risk is High escalated to Malicious — driven by the independent catalog data, not the alert's own severity. That corroboration is the point of the verdict logic.

TEST 6

The two investigations

Steps

1. In the test 5 run, open the item investigation summary.
2. Open the device investigation summary and its risky-items table.
3. Optionally run the device investigation on its own:
4. `KOI - Investigate Device, device_id=<id>`

What to expect

- ✓ Item: catalog risk and AI summary, org exposure, endpoints and users, blocklist state, remediation and approval history.
- ✓ Device: everything installed on the host, which of it is risky, host remediations.

! Known cosmetic issue: a few item-investigation fields render with array brackets — ["high"] rather than high — because the platform resolves arrays differently inside a sub-playbook. The values are correct and the analyst-facing triage summary renders clean.

TEST 7 · SAFETY-CRITICAL

Response stays gated on a human

Steps

1. Let a Malicious verdict reach the response step, or run it directly:
2. `KOI - Block and Remediate`
3. `item_id=<id>  marketplace=<mp>  auto_block=false`
4. Do not approve yet.
5. Inspect the run state and the war room.
6. Then approve, and re-inspect.

What to expect

- ✓ The run parks — runStatus: waiting.
- ✓ koi-blocklist-items-add has NOT executed.
- ✓ The approval shows the full investigation.
- ✓ An already-blocklisted item short-circuits as Already blocked.
- ✓ Only then does the blocklist write fire.

! This is the one test that must pass before you use the pack in production. Triage always calls the response playbook with auto_block=false, so a Malicious verdict can never block software on its own.

TEST 8

Scheduled hunt for risky MCP servers

Steps

1. Run KOI - MCP Server Audit manually first.
2. Leave risk_levels at its default:
3. `high,critical`
4. Review what it flags.
5. Then attach it to a time-triggered Job.

What to expect

- ✓ MCP-server inventory is enumerated.
- ✓ Items at or above the threshold are flagged and reported.
- ✓ Runs on your schedule with no alert needed.

! The audit queries koi-inventory-list with view=mcp_servers. Confirm that view matches how your tenant categorises MCP servers — if it returns nothing, check the categorisation before assuming the estate is clean.

TEST 9

Script Runner job

Steps

1. Confirm the prerequisites exist and match by name.
2. Create the JSON List named exactly:
3. `Koi Script Runner`
4. Automation → Jobs → Run now.
5. Open the run, then open Action Center.

What to expect

- ✓ Work Plan is green.
- ✓ ScriptResult ok:true plus an action_id.
- ✓ SKIPPED info entries where no connected endpoint matches the OS scope — and no failure email.
- ✓ Action Center shows COMPLETED_SUCCESSFULLY.

! Every binding on the 'Names that must match exactly' slide applies here. The war room records which reference broke, and a SKIPPED entry is not a failure — it means no connected, unisolated endpoint currently matches that OS scope.

IF SOMETHING FAILS

Symptom → cause → fix

403 on inventory, koidex or users	Source IP not accepted by KOI's edge	Run the integration on a Cortex engine with an allowlisted egress IP
Test passes but no events arrive	Fetch events not enabled, or no new KOI activity	Enable it, wait two cycles, check koi-fetch-context-get
Triage cannot find the item	The alert maps the KOI payload elsewhere	Adjust alert_description on KOI - Extract Alert Context
Enrichment empty, playbook still green	Best-effort by design — KOI was unreachable	Verify with !koi-users-list limit=1
Job entry reports SKIPPED	No connected, unisolated endpoint matches the OS scope	Check group membership and endpoint_os in the List
Script not found at dispatch	Library name does not match script.name exactly	Fix the name, or pin script.uuid to be rename-proof

The first row accounts for most real-world failures — check egress before anything else.

SIGN-OFF

One line of evidence per test

1. Test returns Success and koi-devices-list returns rows
2. koi_koi_raw event counts grow between fetch cycles
3. Read commands return tenant data with no 403
4. Dashboard widgets render and XDM fields are populated
5. Triage completes with a verdict and both summaries
6. Item and device investigations carry real KOI data
7. Run parks on approval; blocklist write did not execute
8. MCP audit flags items at or above the threshold
9. Job returns ScriptResult ok with an action_id

All nine green → the pack is ready for production use.

Full detail for every step is in the customer guide — KOI_Integration_Customer_Guide_v1.3.0 (Word and PDF).